Cloud Platforms (AWS)

Session 3: Advanced SQL – Joins, Triggers & Assertions

MIBA24 · Master of Science in Business Analytics

ESADE Business School | February 18, 2026

Oriol Rius Canals | joseporiol.rius@esade.edu

Session Hook

"When business tables talk to each other, insights emerge."

Today we unlock the true power of relational databases

Today's Agenda

JOIN Operations

INNER, LEFT, RIGHT, FULL joins

Triggers & Views

Automation and reusable queries

SQL Optimization

CTEs, subqueries, query plans

Why Advanced SQL Matters

Real Analytics Scenarios

Business questions require combining multiple data sources:

- "What are sales by customer segment?"
- "Which products sell best in each region?"
- "Who are our top 10 customers by revenue?"
- "What's the average order value trend?"

Key insight: These questions need data from multiple tables — JOINs make it possible.

Data Integrity & Automation

Advanced SQL also enables governance:

- ✓ **Triggers** — Automate data validation
- ✓ **Views** — Reusable query templates
- ✓ **Constraints** — Enforce business rules
- ✓ **CTEs** — Readable, maintainable queries

Goal: Transform raw data into analytical assets

Quick Check-In

From Last Week

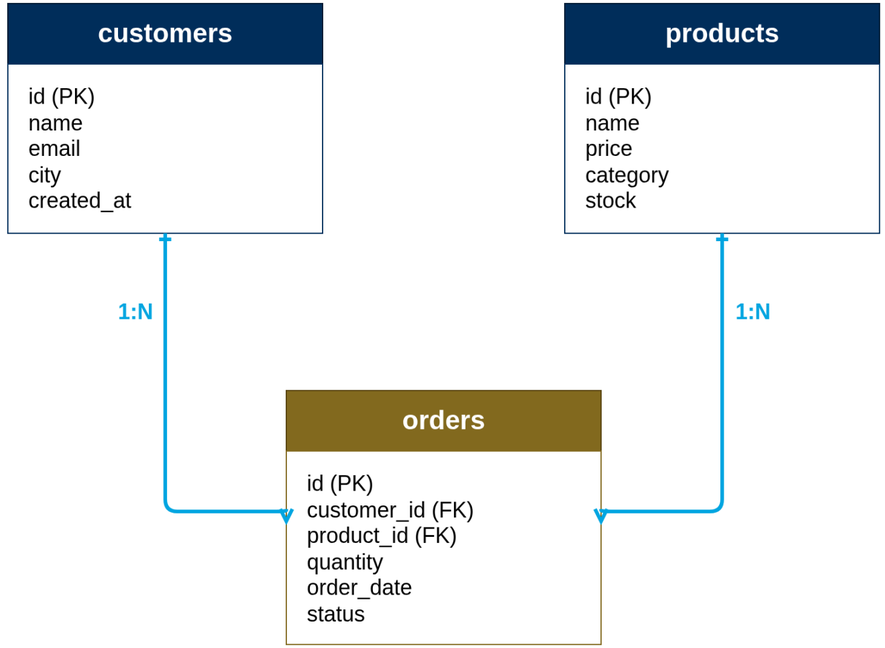
1. **RDS Instance Running?**
   PostgreSQL database accessible
2. **Schema Created?**
   customers, products, orders tables
3. **SQL Scripts Saved?**
   CREATE and INSERT statements versioned

Quick Polls

- Raise hand: RDS connection working
- Raise hand: Practiced CRUD this week
- Raise hand: Need help with setup

RDS not running?
Start it now — we need it for today's exercises!

Recall: Our Demo Schema



✕ Single-table queries can't answer: "Which customer bought which product?"

✓ **Solution:** JOINs combine tables via foreign keys

Part 1: JOIN Operations

Combining data from multiple tables

What is a JOIN?

A JOIN combines rows from two or more tables based on a related column

Two Tables

```
-- customers table
| id | name       | city       |
|----|------------|------------|
| 1  | Ana        | Barcelona  |
| 2  | John       | London     |

-- orders table
| id | customer_id | amount |
|----|-------------|--------|
| 1  | 1           | 100    |
| 2  | 1           | 200    |
```

Combined Result

```
-- After JOIN on customer_id = id
| name | city       | amount |
|------|------------|--------|
| Ana  | Barcelona  | 100    |
| Ana  | Barcelona  | 200    |
```

Customer info + Order info in one result!

The Four JOIN Types

INNER JOIN

Only matching rows from both tables. The most common type.

✓ Use when you need complete data only

LEFT JOIN

All rows from left table, matching rows from right (NULL if no match).

✓ Use to find "customers with or without orders"

RIGHT JOIN

All rows from right table, matching rows from left (NULL if no match).

✓ Less common — usually rewrite as LEFT JOIN

FULL OUTER JOIN

All rows from both tables, NULLs where no match exists.

✓ Use for data reconciliation and audits

Rule of thumb: Start with INNER JOIN. Use LEFT JOIN when you need "all X, even without Y". FULL OUTER is rare in analytics.

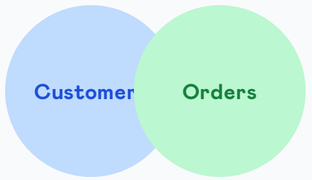
INNER JOIN: The Default

Syntax

```sql
SELECT
  customers.name,
  customers.city,
  orders.quantity,
  products.name AS product_name,
  products.price
FROM orders
INNER JOIN customers
  ON orders.customer_id = customers.id
INNER JOIN products
  ON orders.product_id = products.id;
```

Note: `INNER` is optional — `JOIN` alone means INNER JOIN

Visual Explanation



INNER JOIN = The overlap (purple area)

Result

Only customers who have placed orders

Customers without orders are excluded

LEFT JOIN: Include All from Left

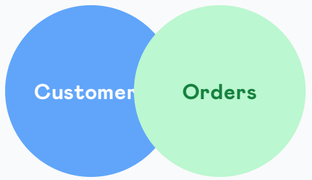
Syntax

```sql
SELECT
  customers.name,
  customers.city,
  COUNT(orders.id) AS order_count
FROM customers
LEFT JOIN orders
  ON customers.id = orders.customer_id
GROUP BY customers.id, customers.name, customers.city;
```

Result includes:

name	city	order_count
Ana	Barcelona	3
John	London	1
Marie	Paris	**0**

Visual Explanation



← LEFT JOIN = All blue + overlap

When to Use

- ✓ Find customers who haven't ordered
- ✓ Count orders per customer (including 0)
- ✓ Identify inactive records

FULL OUTER JOIN: Everything

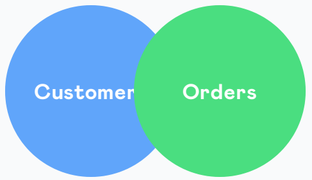
Syntax

```sql
SELECT
  customers.name,
  orders.id AS order_id,
  orders.quantity
FROM customers
FULL OUTER JOIN orders
  ON customers.id = orders.customer_id;
```

Result includes NULLs on both sides:

name	order_id	quantity
Ana	1	5
NULL	99	10
Marie	**NULL**	**NULL**

Visual Explanation



↔ FULL OUTER = Everything from both

When to Use

- ✓ Data reconciliation between systems
- ✓ Find orphan records
- ✓ Audit and data quality checks

JOIN Best Practices

Write efficient, readable JOIN queries

Do's

Good Practices

- **Use table aliases** — `FROM orders o JOIN customers c`
- **Be explicit** — Write `INNER JOIN` not just `JOIN`
- **Prefix columns** — `c.name` not just `name`
- **Index JOIN columns** — Foreign keys need indexes

Don'ts

Avoid These

- **Cartesian products** — Forgetting the ON clause
- **Too many JOINs** — 4+ often signals poor schema
- **SELECT * with JOINs** — Ambiguous columns
- **Ignoring NULLs** — LEFT JOIN needs COALESCE

Pro tip: Always test your JOINs on small datasets first to verify you get expected row counts.

Live Demo: Multi-Table Queries

Query Examples

```sql
-- Orders with customer and product info
SELECT
  c.name AS customer,
  p.name AS product,
  o.quantity,
  p.price,
  (o.quantity * p.price) AS total
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id
INNER JOIN products p ON o.product_id = p.id
ORDER BY total DESC;
```

```sql
-- Customers with no orders
SELECT c.name, c.email
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.id IS NULL;
```

Practice Tasks

1. **List all orders with details** — Customer name, product, quantity, total
2. **Find customers without orders** — Use LEFT JOIN + IS NULL
3. **Products never ordered** — Inventory optimization insight

Part 2: Aggregations & Grouping

Summarizing data for business insights

Aggregate Functions

The Big Five for summarizing data

COUNT(*)

Number of rows in a group

SUM(column)

Total of numeric values

AVG(column)

Average of numeric values

MIN(column)

Smallest value in group

MAX(column)

Largest value in group

Key Insight

Aggregates collapse many rows into one summary row

Example: `SELECT COUNT(*), SUM(quantity), AVG(price) FROM orders`

GROUP BY: Aggregation by Category

Concept

GROUP BY divides data into groups, then aggregates each group separately

```sql
SELECT
  city,
  COUNT(*) AS customer_count
FROM customers
GROUP BY city;
```

city	customer_count
Barcelona	12
London	8
Paris	5

Business Analytics Examples

```sql
-- Sales by product category
SELECT
  p.category,
  COUNT(o.id) AS orders,
  SUM(o.quantity * p.price) AS revenue
FROM orders o
JOIN products p ON o.product_id = p.id
GROUP BY p.category
ORDER BY revenue DESC;
```

```sql
-- Monthly order trend
SELECT
  DATE_TRUNC('month', order_date) AS month,
  COUNT(*) AS orders,
  SUM(quantity) AS items_sold
FROM orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;
```

Rule: Every non-aggregated column in SELECT must appear in GROUP BY

HAVING: Filter Aggregated Results

WHERE vs HAVING

WHERE

Filters **individual rows** BEFORE grouping

HAVING

Filters **groups** AFTER aggregation

Memory trick: WHERE = before GROUP BY, HAVING = after GROUP BY

Example

```sql
-- Customers with more than 5 orders
SELECT
  c.name,
  c.city,
  COUNT(o.id) AS order_count,
  SUM(o.quantity * p.price) AS total_spent
FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN products p ON o.product_id = p.id
WHERE o.status = 'completed'    -- Filter rows
GROUP BY c.id, c.name, c.city
HAVING COUNT(o.id) > 5          -- Filter groups
ORDER BY total_spent DESC;
```

Query flow:

1 FROM → 2 WHERE → 3 GROUP BY → 4 HAVING → 5 SELECT → 6 ORDER BY

10-Minute Break

Stretch, grab coffee, let the JOINs sink in

Back at **10:45** sharp

Next: Subqueries, CTEs, and automation with triggers

Part 3: Subqueries & CTEs

Writing readable, maintainable complex queries

Subqueries: Queries Inside Queries

What is a Subquery?

A query nested inside another query. Can appear in:

- ✓ **SELECT** — Compute a value
- ✓ **FROM** — Create derived table
- ✓ **WHERE** — Filter by subquery result

Pitfall Warning

Overusing nested queries leads to:

- ✗ Hard to read code
- ✗ Poor performance
- ✗ Difficult debugging

Examples

```sql
-- Subquery in WHERE
SELECT name, email
FROM customers
WHERE id IN (
  SELECT DISTINCT customer_id
  FROM orders
  WHERE order_date > '2026-01-01'
);
```

```sql
-- Subquery in SELECT (scalar)
SELECT
  name,
  (SELECT COUNT(*)
   FROM orders
   WHERE customer_id = customers.id) AS order_count
FROM customers;
```

Rule: If a subquery runs once per row (correlated), consider a JOIN instead

CTEs: Common Table Expressions

What is a CTE?

A temporary named result set defined with `WITH`

- ✓ More readable than nested subqueries
- ✓ Can reference multiple times
- ✓ Easier to debug step by step
- ✓ Self-documenting code

Think of CTEs as: Named subqueries that make complex logic readable

Syntax

```sql
WITH customer_totals AS (
  SELECT
    customer_id,
    SUM(quantity * price) AS total_spent
  FROM orders o
  JOIN products p ON o.product_id = p.id
  GROUP BY customer_id
),
high_value_customers AS (
  SELECT customer_id
  FROM customer_totals
  WHERE total_spent > 1000
)
SELECT
  c.name,
  c.email,
  ct.total_spent
FROM customers c
JOIN high_value_customers hvc ON c.id = hvc.customer_id
JOIN customer_totals ct ON c.id = ct.customer_id
ORDER BY ct.total_spent DESC;
```

Subquery vs CTE

Same logic, different readability

Nested Subqueries

Hard to Follow

```sql
SELECT name
FROM customers
WHERE id IN (
  SELECT customer_id
  FROM orders
  WHERE product_id IN (
    SELECT id
    FROM products
    WHERE category = 'Electronics'
  )
  AND order_date > '2026-01-01'
);
```

- ✗ Hard to follow
- ✗ Difficult to debug
- ✗ Can't reuse parts

With CTEs

Clear Step-by-Step

```sql
WITH electronics AS (
  SELECT id FROM products
  WHERE category = 'Electronics'
),
recent_orders AS (
  SELECT DISTINCT customer_id
  FROM orders
  WHERE product_id IN (SELECT id FROM electronics)
    AND order_date > '2026-01-01'
)
SELECT name
FROM customers
WHERE id IN (SELECT customer_id FROM recent_orders);
```

- ✓ Clear step-by-step
- ✓ Easy to debug each CTE
- ✓ Reusable named sets

Window Functions: Analytics Powerhouse

What are Window Functions?

Perform calculations across a set of rows related to the current row — without collapsing them

- ✓ Running totals
- ✓ Rankings
- ✓ Moving averages
- ✓ Comparisons to previous/next row

Key difference: GROUP BY collapses rows. Window functions keep all rows.

Examples

```sql
-- Rank customers by total spending
SELECT
  c.name,
  SUM(o.quantity * p.price) AS total_spent,
  RANK() OVER (ORDER BY SUM(o.quantity * p.price) DESC) AS rank
FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN products p ON o.product_id = p.id
GROUP BY c.id, c.name;
```

```sql
-- Running total by date
SELECT
  order_date,
  quantity,
  SUM(quantity) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

Part 4: Triggers, Views & Automation

Enforcing data integrity and creating reusable analytics

Views: Saved Queries

What is a View?

A stored query that acts like a virtual table

- ✓ Simplify complex queries
- ✓ Provide consistent interface
- ✓ Control data access
- ✓ Support KPI dashboards

Think of views as: Reusable query templates that always show fresh data

Creating a View

```sql
-- Create a customer summary view
CREATE VIEW customer_summary AS
SELECT
  c.id,
  c.name,
  c.city,
  COUNT(o.id) AS total_orders,
  COALESCE(SUM(o.quantity * p.price), 0) AS total_revenue,
  MAX(o.order_date) AS last_order_date
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
LEFT JOIN products p ON o.product_id = p.id
GROUP BY c.id, c.name, c.city;
```

```sql
-- Use it like a table
SELECT * FROM customer_summary
WHERE total_revenue > 500
ORDER BY total_revenue DESC;
```

KPI Dashboard View Example

Business KPIs View

```sql
CREATE VIEW daily_kpis AS
WITH daily_metrics AS (
  SELECT
    DATE(order_date) AS date,
    COUNT(*) AS orders,
    COUNT(DISTINCT customer_id) AS unique_customers,
    SUM(quantity) AS items_sold
  FROM orders
  WHERE status = 'completed'
  GROUP BY DATE(order_date)
)
SELECT
  date,
  orders,
  unique_customers,
  items_sold,
  ROUND(items_sold::numeric / orders, 2) AS avg_items_per_order
  LAG(orders) OVER (ORDER BY date) AS prev_day_orders,
  orders - LAG(orders) OVER (ORDER BY date) AS order_change
FROM daily_metrics
ORDER BY date DESC;
```

Sample Output

date	orders	customers	items	change
2026-02-18	23	18	67	+4
2026-02-17	19	15	52	-2
2026-02-16	21	17	58	+3

Benefits

- ✓ Dashboard queries become simple
- ✓ Logic is centralized
- ✓ Easy to modify and extend
- ✓ Analysts don't need complex SQL

Triggers: Automated Database Actions

What is a Trigger?

Code that runs automatically when data changes

- → BEFORE INSERT/UPDATE/DELETE
- → AFTER INSERT/UPDATE/DELETE
- → INSTEAD OF (for views)

Use Cases

✓ Audit logging

✓ Update inventory on order

✓ Enforce business rules

✓ Maintain computed columns

```
-- Function to update product stock
CREATE OR REPLACE FUNCTION update_stock()
RETURNS TRIGGER AS $$
BEGIN
  -- Decrease stock when order is placed
  UPDATE products
  SET stock = stock - NEW.quantity
  WHERE id = NEW.product_id;

  -- Check if stock went negative
  IF (SELECT stock FROM products WHERE id = NEW.product_id) < 0
    RAISE EXCEPTION 'Insufficient stock for product %', NEW.pro
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Attach trigger to orders table
CREATE TRIGGER trigger_update_stock
AFTER INSERT ON orders
FOR EACH ROW
EXECUTE FUNCTION update_stock();
```

Constraints & Assertions

Built-in Constraints

PRIMARY KEY — Unique identifier, not NULL

FOREIGN KEY — Referential integrity

UNIQUE — No duplicate values

CHECK — Custom validation rule

NOT NULL — Value required

Note:
Standard SQL ASSERTION (cross-table constraints) is not supported in PostgreSQL. Use triggers instead.

CHECK Constraint Examples

```sql
-- Ensure positive values
ALTER TABLE products
ADD CONSTRAINT positive_price
CHECK (price > 0);

ALTER TABLE products
ADD CONSTRAINT positive_stock
CHECK (stock >= 0);

-- Ensure valid status values
ALTER TABLE orders
ADD CONSTRAINT valid_status
CHECK (status IN ('pending', 'processing',
                  'shipped', 'completed', 'cancelled'));

-- Ensure quantity is reasonable
ALTER TABLE orders
ADD CONSTRAINT reasonable_quantity
CHECK (quantity > 0 AND quantity <= 1000);
```

Live Demo: Building KPI Infrastructure

What We'll Build

1. **Customer Summary View** — Orders, revenue, last activity
2. **Daily KPIs View** — Orders, customers, items, trends
3. **Stock Update Trigger** — Auto-decrement on order
4. **CHECK Constraints** — Validate prices, quantities, status

End Result

An Enriched Data Model

- ✓ Complex analytical queries simplified
- ✓ Automated stock management
- ✓ Data integrity enforced
- ✓ Dashboard-ready KPIs

Key Takeaways

JOINs Connect Data

INNER for matches, LEFT for all-plus-matches. Foreign keys are the bridges.

Aggregations Summarize

GROUP BY + HAVING for dimensional analysis. Window functions for rankings.

CTEs Over Subqueries

WITH clause makes complex queries readable and maintainable.

Views Simplify Access

Store complex queries as virtual tables. Perfect for KPI dashboards.

Triggers Automate

Enforce business rules automatically. Update related data on changes.

Constraints Protect

CHECK constraints validate data at the database level.

What We Built Today

KPI Views

Customer summary and daily metrics ready for dashboards

Stock Trigger

Automatic inventory updates on order placement

Data Constraints

Business rules enforced at the database level

🏆 **Achievement Unlocked:** You've built an enriched data model capable of complex analytical queries!

Common Pitfalls to Avoid

Learn from common mistakes

Do's

Good Practices

- Use CTEs for complex queries
- Index JOIN columns
- Specify columns, not SELECT *
- Test triggers thoroughly
- Use EXPLAIN ANALYZE

Don'ts

Avoid These

- Overusing nested queries
- Confusing redundancy with denormalization
- Ignoring query plans
- Too many triggers
- Missing indexes on foreign keys

Remember: Prevention beats correction. Design for performance from the start.

References & Resources

Joining Data in SQL

INNER, LEFT, OUTER, CROSS joins + set theory (UNION, INTERSECT) + subqueries

datacamp.com

Data Manipulation in SQL

CTEs, window functions, CASE statements, advanced subqueries

datacamp.com

Thank You!

See you next class!

Session 4: Containers & Docker – Image Crafting, ECR & ECS

Questions? → joseporiol.rius@esade.edu